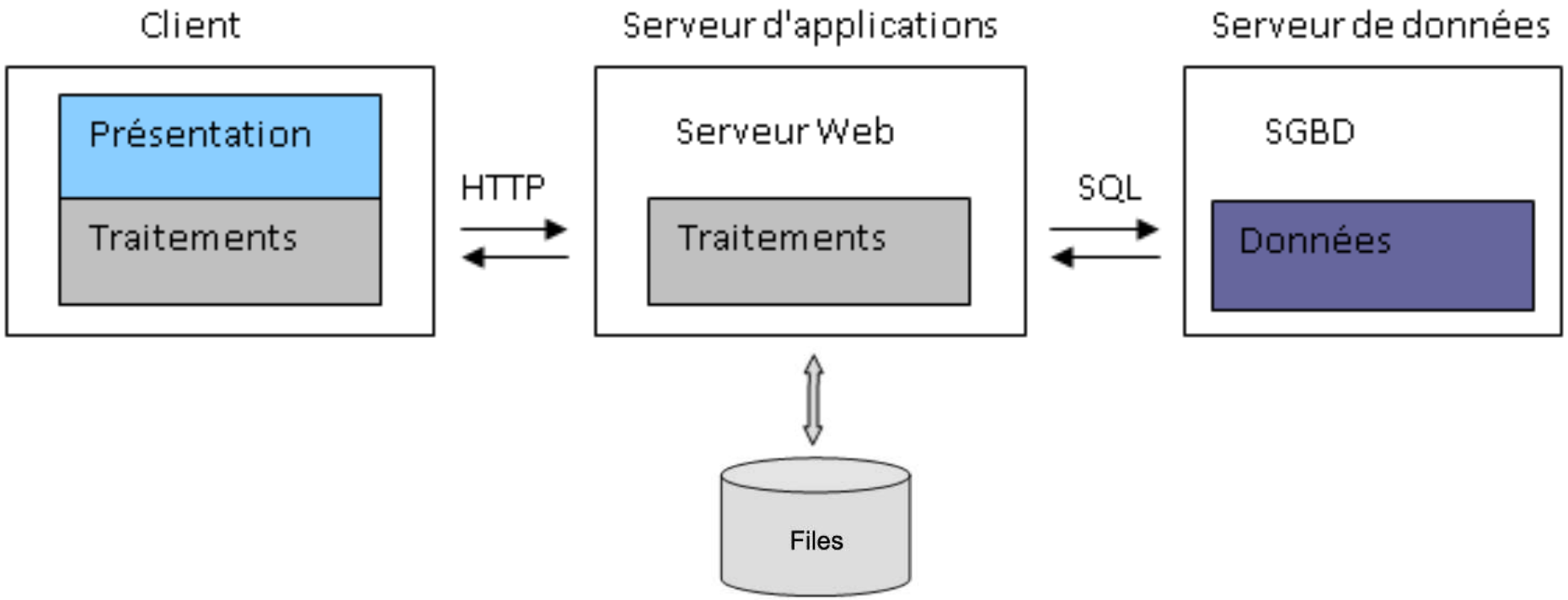
PROGRAMMATION WEB DYNAMIQUE

Benoit Gaudou - benoit.gaudou@ut-capitole.fr

Source : Laurent Perrussel

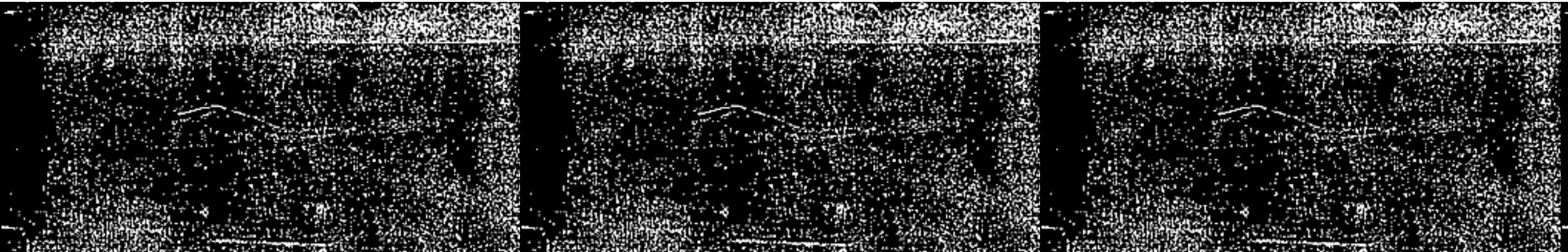
FRONTEND VS BACKEND



FrontEnd
HTML,CSS,Javascript

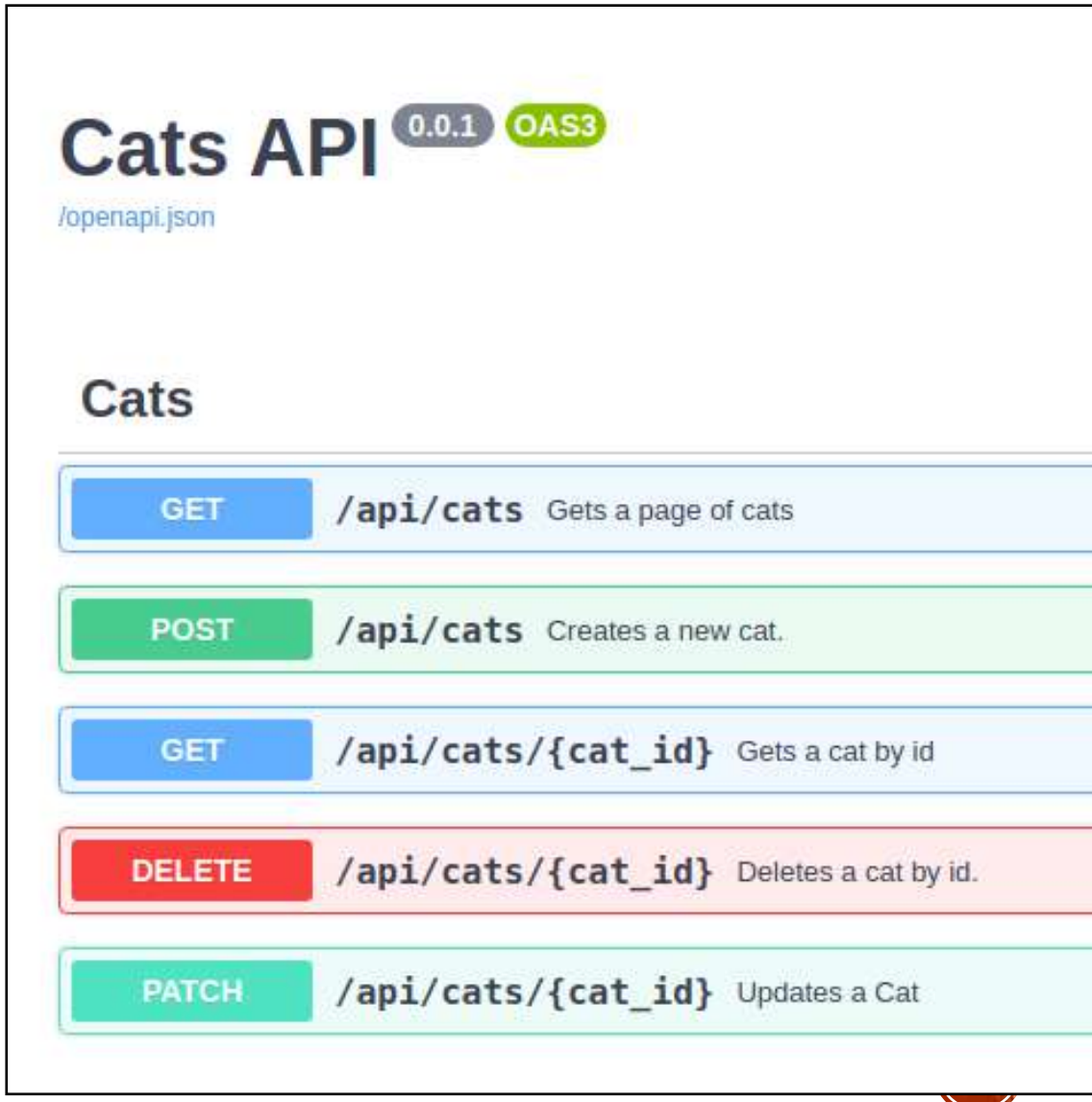
BackEnd
PHP, Node.JS, Python (Flask, Django),
Java (Springboot)…

WEB SERVICE



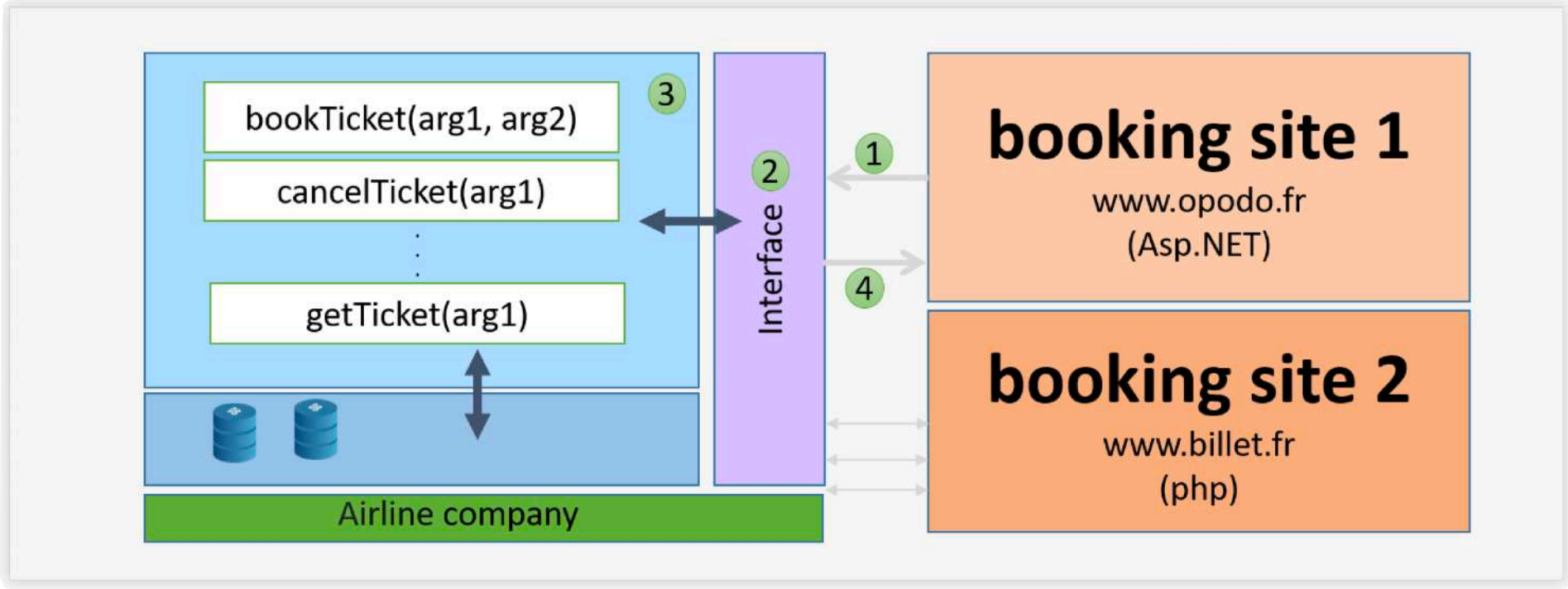
WEB SERVICES - DEFINITIONS

- Un **Web Service** est une **entité logicielle** qui **expose** des fonctionnalités au **consommateur**
  - Sur internet ou intranet
  - Au travers de **fonctions qui peuvent être appelées via internet**
- **Basé sur les standards du Web**
  - HTTP, XML, SOAP, WSDL, UDDI,…
- Peut être be implémenté dans n'importe **quel langage** et plate-forme
- Les web services sont des applications **autonomes**, **modulaires**, **distribuées** et **dynamiques**



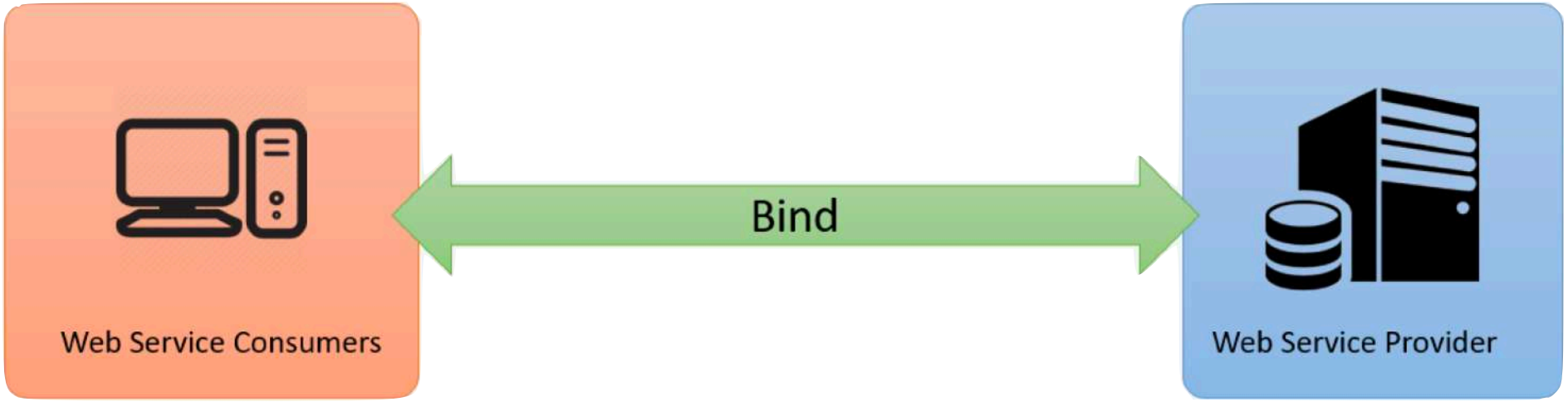
COMMENT ÇA MARCHE ?

- **1 :** Le client envoie une requête au serveur
- **2 :** Le serveur décode la requête et extrait la fonction à appeler
- **3 :** La fonction est exécutée
- **4 :** La réponse est formatée et renvoyé au client



WEB SERVICES - ARCHITECTURE

- Les plus simples Web service ont 2 participants:
  - Le **producteur** (fournisseur), qui présente un interface et une implémentation du service
  - Le **consommateur** (consumer) qui utilise le Web service



WEB SERVICES - CARACTÉRISTIQUES

- Les Web services utilisent **XML/JSON** pour échanger des messages entre le fournisseur et le consommateur.

- **Exemple d'XML**

```
<employees>
    <employee>
      <firstName>John</firstName> <lastName>Doe</lastName>
    </employee>
    <employee>
      <firstName>Anna</firstName> <lastName>Smith</lastName>
    </employee>
    <employee>
      <firstName>Peter</firstName> <lastName>Jones</lastName>
    </employee>
</employees>
```

- **Exemple de JSON (JSON = JavaScript Object Notation):**

```
{"employees":[
    { "firstName":"John", "lastName":"Doe" },
    { "firstName":"Anna", "lastName":"Smith" },
    { "firstName":"Peter", "lastName":"Jones" }
]}
```

WEB SERVICES - CARACTÉRISTIQUES

- Les Web services utilisent **XML/JSON** pour échanger des messages entre le fournisseur et le consommateur.

- Le producteur du web service s'engage sur les **stabilité des services exposés**, mais **l'implémentation derrière peut évoluer** sans compromettre la capacité du client à interagir avec le service.

- Les services web permettent aux clients d'invoquer des procédures, des fonctions et des méthodes sur des objets distants en utilisant des protocoles standard du web.

WEB SERVICES - BÉNÉFICES

- **Exposition d'une fonction sur le réseau**
  - Les web services exposent les fonctionnalités d'un code existant sur le réseau.
- **Mise à disposition de données**
  - Les web services permettent de rendre disponible des données avec un protocole uniforme.
- **Interopérabilité**
  - Les web services permettent à diverses applications de communiquer entre elles et de partager des données et des services.
- **Polyvalence**
  - Les web services peuvent être développés sur n'importe quelle plate-forme et dans n'importe quel langage.
- **Flexibilité**
  - Les codes des web services peuvent être réutilisés.
- **Réduction des coûts**
  - Les web services utilisent des normes et des protocoles ouverts pour la communication.

2 PRINCIPAUX TYPES DE WEB SERVICES

- **Simple Object Access Protocol (SOAP)**
  - Très lourd, le plus ancien, basé sur des échanges au travers de fichiers XML complexes.
- **REpresentational State Transfer (REST)**
  - Repose sur des URIs et le protocole **HTTP**
  - Utilisation des 5principaux verbes HTTP : POST, HEAD, GET, PUT, DELETE
    - --> proche du concept de CRUD
  - Echange de données au format XML, JSON ou les deux

Create, Read, Update, and Delete (**CRUD**) : les 4 principales actions sur une BD.

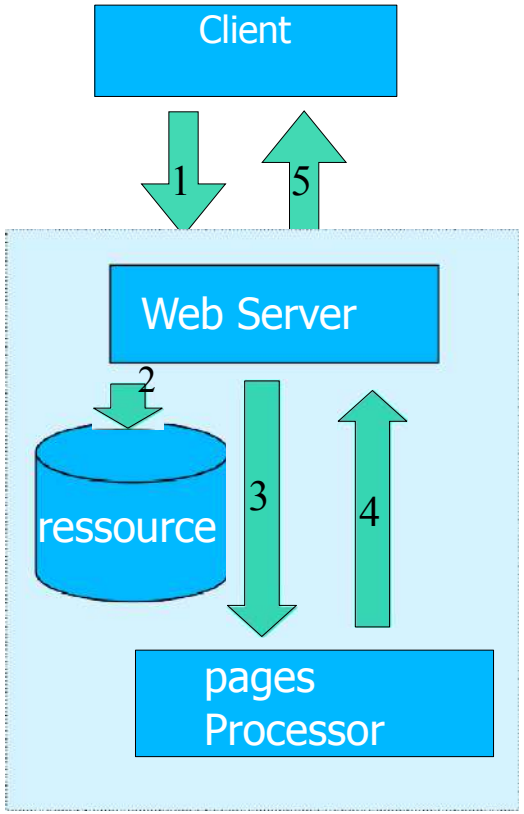
CRUD	SQL	HTTP
create	INSERT	PUT
read	SELECT	GET
update	UPDATE	PUT
delete	DELETE	DELETE

2 PRINCIPAUX TYPES DE WEB SERVICES

- Nous utiliserons **l'architecture de Web service REST**.
- Nous ajouterons un système de **templating** afin de produire nos pages web dynamiques.

GENERATION D'UN SITE

1. Réception d'une requête HTTP
2. Le web service parse la méthode HTTP et l'**achemine** vers la fonction appropriée (**routing**)
3. La fonction récupère les paramètres de la requête, exécute le code, en accédant à des sources de données … et transmet ces données au moteur de rendu de page (*page renderer*) (**templating**)
4. Le moteur génère la page HTML
5. La page générée est renvoyée au consommateur.




```
GET /customers/43456 HTTP/1.1
Host: www.example.org
```

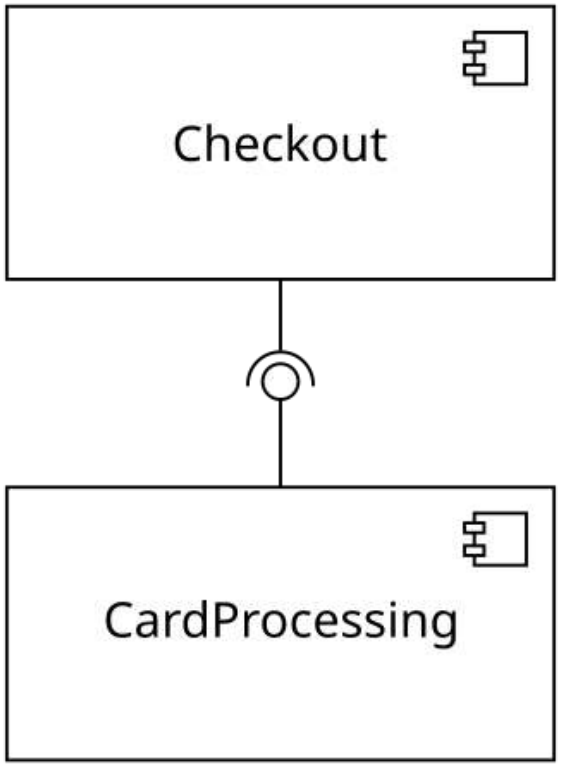
SPRING FRAMEWORK

Introduction: XML, JSON parsing

INTRODUCTION AU FRAMEWORK SPRING

- **Dans le monde Java, ce type de développement se base sur JavaEE et repose sur une approche à base de composants, qui vise**
  - Separation des responsabilités
  - Amélioration de la productivité
  - Indépendence de la plate-forme d'exécution

- Le framework **Spring: vise à fournir les mêmes fonctionnalités dans une approche plus légère**
  - Accès aux données
  - Web
  - Intégration

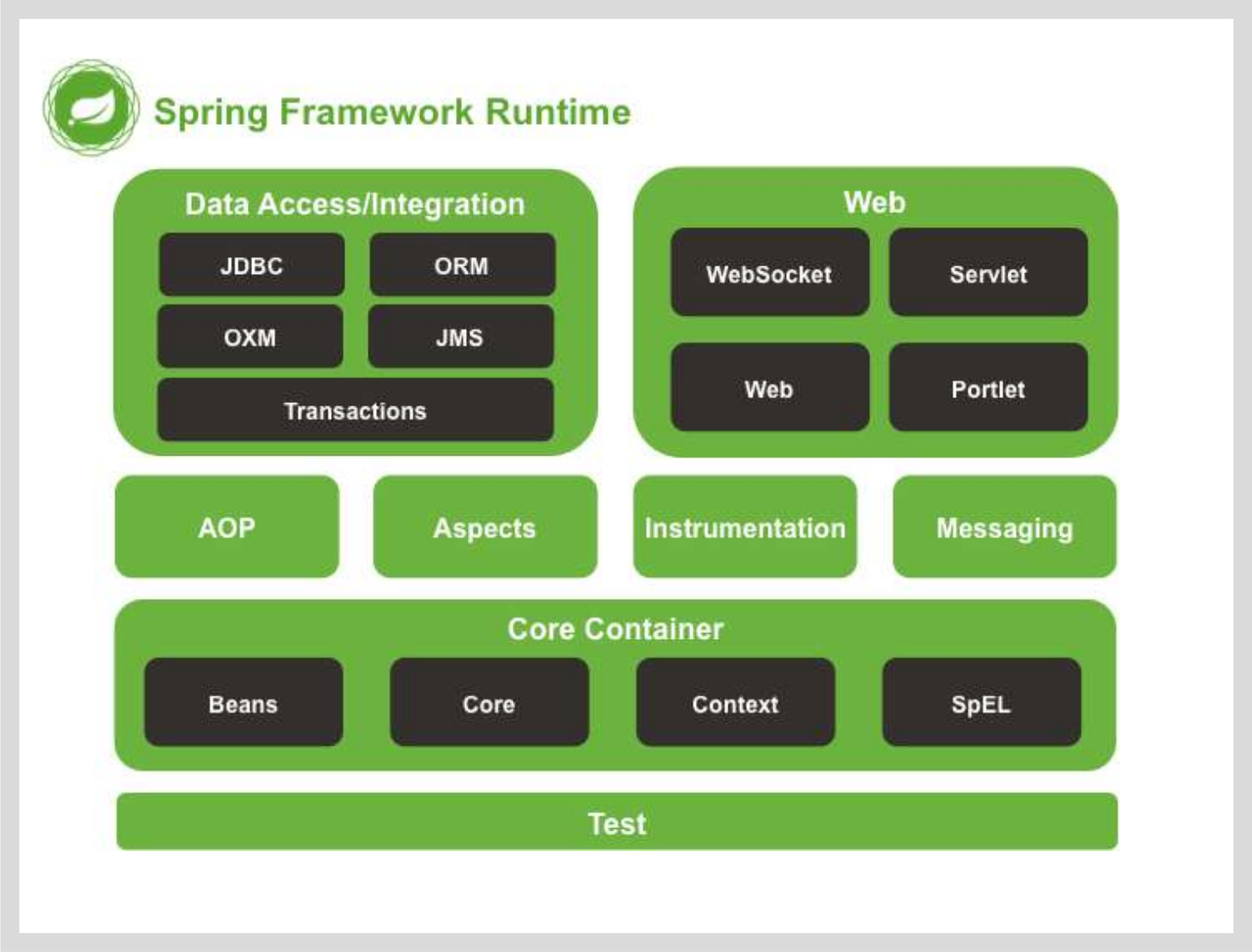


LES BUTS AFFICHÉS PAR L'ÉQUIPE DE SPRING GOALS

- *J2EE devrait être plus facile à utiliser*
- *La gestion des exceptions est sur-utilisée en Java, un framework ne devrait pas vous forcer à rattraper des exceptions que vous avez peu de chances de pouvoir traiter …*
- *Spring devrait être un plaisir à utiliser*

- ***Spring n'est pas en compétition avec les bonnes solutions/librairies existantes, but devrait les intégrer.*** *(Par exemple, JDO, Toplink, et Hibernate* ***sont de bonnes solutions d'ORM*** *(Object/Relational Mapping))*

SPRING FRAMEWORK MODULES OVERVIEW



WEBSERVICES (REST) WITH SPRING

- Spring vous permet également de développer des interfaces graphiques web et des **API REST** :
  - **Routing**
  - Contrôleurs POJO (Plain Old Java Object)
  - Abstraction en ce qui concerne la requête et la réponse HTTP.
  - Sérialisation/désérialisation (par exemple JSON<=>Java)
  - Gestion des exceptions.

  - **Templating** : Couplage avec les vues jsp, velocity, freemarker …

SPRINGBOOT

- **Spring Boot** est un moyen rapide de créer une application Spring prête à être mise en production.
  - Il est basé sur le Framework Spring
  - Il est conçu pour vous permettre d'être opérationnel le plus rapidement possible.
  - Il evite les configurations complexes de l'application (souvent faites en XML)

- **SpringBoot = Spring avec une configuration par défaut**

SPRINGBOOT EN PRATIQUE

Introduction: XML, JSON parsing

APPLICATION PRINCIPALE

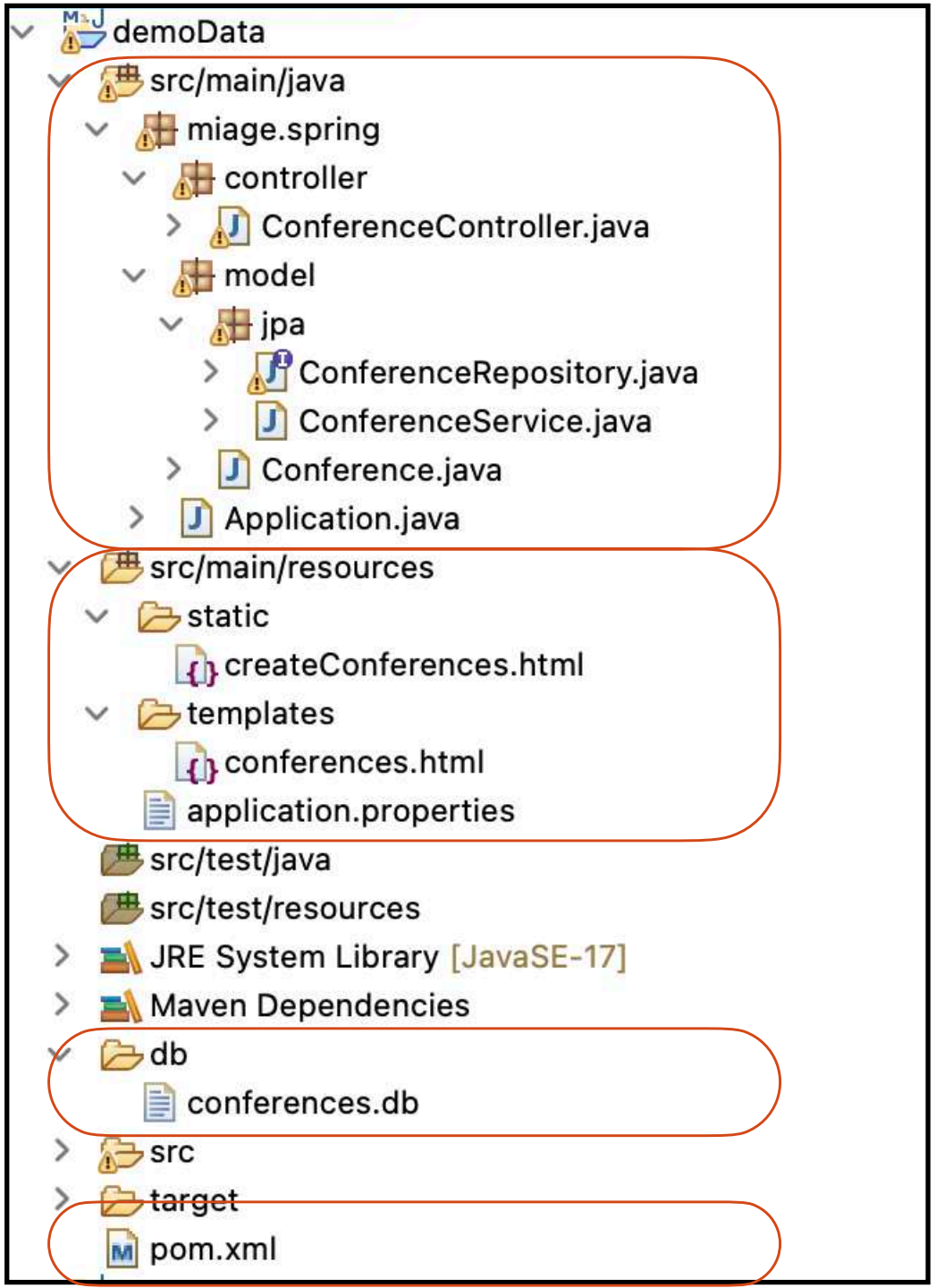
- On peut lancer une application SpringBoot avec la simple classe suivante (et quelques dépendances MAVEN):
  cela va exécuter un serveur web, à partir d'une configuration par défaut.

```
@SpringBootApplication
public class Application {
  public static void main(String[] args)     {
      SpringApplication.run(Application.class,args);
   }
}
```

- Cela lance un serveur web accessible à l'adresse http://localhost:8080 (par défaut)

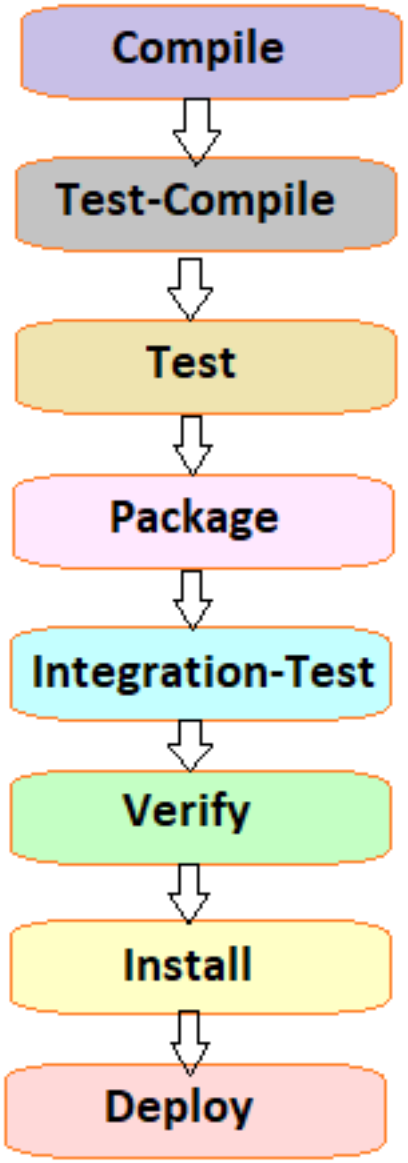
ARCHITECTURE D'UN PROJET SPRINGBOOT

- Un projet SpringBoot contiendra :
  - Le **code Java**
    `src/main/java`
  - Les **ressources** (pages web static, templates, configuration…)
    `src/main/resources`
  - Le fichier **pom.xml** (à la racine)
  - Autres ressources (db…)



MAVEN

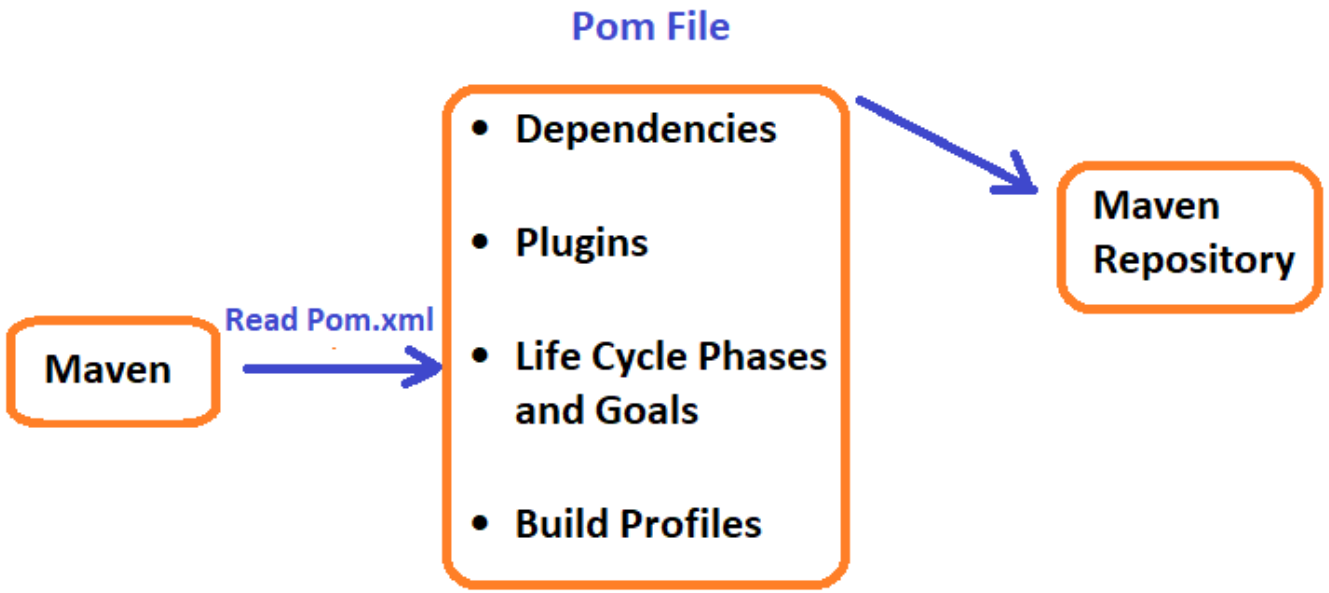
- Maven est un puissant **outil de gestion de projet** qui simplifie le processus de construction des projets basés sur Java tout au long de leur cycle de vie.
- Nous l'utiliserons principalement pour sa **gestion des packages**.

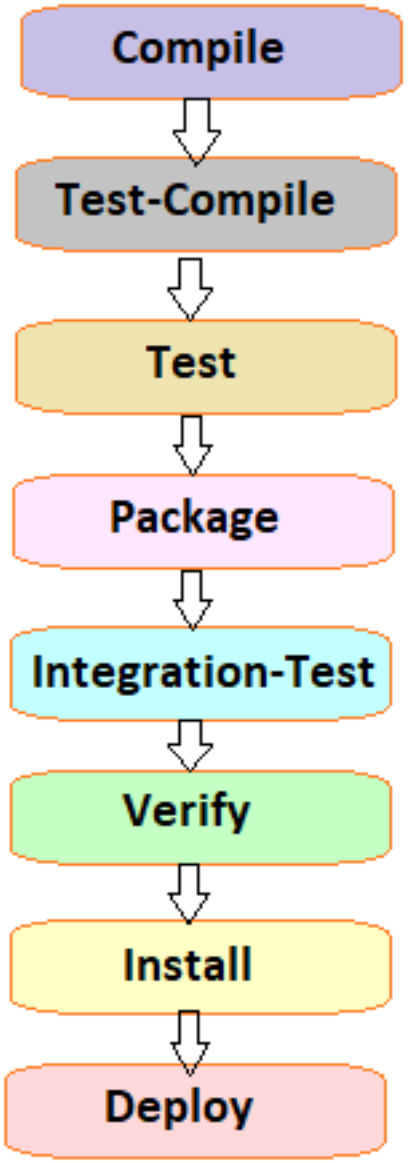


https://geekflare.com/fr/apache-maven-for-beginners/

MAVEN

- Nous l'utiliserons principalement pour sa **gestion des packages**.
  - Tout est déclaré dans le fichier pom.xml.
  - Les fichiers téléchargés sont placés dans le dossier .m2 (dossier caché) de votre ordinateur





https://geekflare.com/fr/apache-maven-for-beginners/

APPLICATION PRINCIPALE

- Pour que le code fonctionne, vous avez besoin de 2 éléments principaux dans le pom.xml (dans <project>).
  - Avoir comme parent POM Spring Boot starter.

```
<parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
      <version>2.4.3</version>
      <relativePath/> <!-- lookup parent from repository -->
</parent>
```

  - Inclure la dependence spring-boot starter web.

```
<dependencies>
   <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

Exemple du fichier pom.xml complet : https://spring.io/guides/gs/spring-boot/

RAPPEL: JAVA ANNOTATIONS

JAVA ANNOTATIONS

- Introduction officielle Oracle Java Doc

- *Les annotations sont une forme de **métadonnées**, qui fournissent des données sur un programme mais qui ne font pas partie du programme lui-même. Les annotations n'ont pas d'effet direct sur le fonctionnement du code qu'elles annotent.*
- *Les annotations ont un certain nombre d'utilisations, parmi lesquelles*
  - ***Informations pour le compilateur** - Les annotations peuvent être utilisées par le compilateur pour détecter des erreurs ou supprimer des avertissements.*
  - ***Traitement au moment de la compilation et du déploiement** - Les outils logiciels peuvent traiter les informations d'annotation pour générer du code, des fichiers XML, etc.*
  - ***Traitement au moment de l'exécution** - Certaines annotations peuvent être examinées au moment de l'exécution.*

JAVA ANNOTATIONS

- Pour le compilateur :

```
@Override
void mySuperMethod() { ... }
```

- Pour le compilateur, avec paramètres :

```
@SuppressWarnings(value = "unchecked")
void myMethod() { ... }
```

JAVA ANNOTATIONS

- Annotations standards Java (JSR250, JSR 330, JSRxxx, …

```
@PostContruct
void myDestroyMethod() { ... )
```

```
@Entity  // indicate that this class is an entity to be managed by JPA
public class myEntity { ... }
```

- Junit exemples :

```
@BeforeMethod // indicate that this method should be run before the tests void
myInitMethod() { ... }
```

```
@Test
public void mySuperTest() { ... }
```

- Spring exemples :

```
@Component
public class mySuperComponent { ... }
```

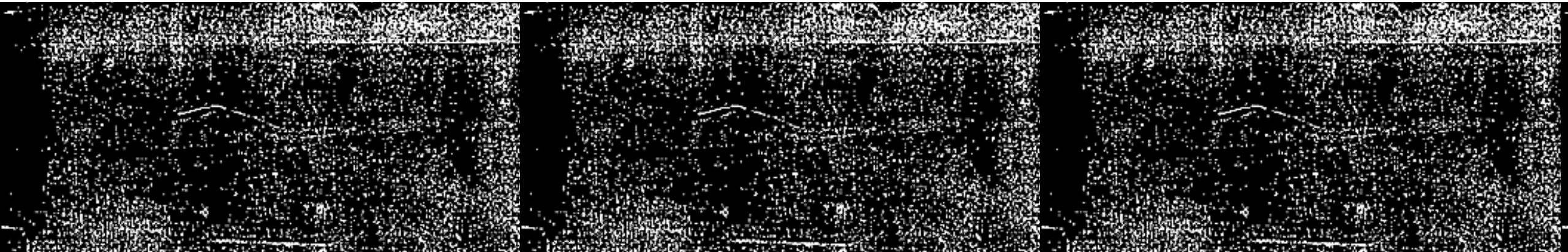
ANNOTATION: SYNTAXE

- Une annotation peut avoir 1 ou plusieurs paramètres :

```
@RequestParam("name")
@RequestParam(value = "name", defaultValue = "test")
```

- Avec 1 paramètre, seule la valeur est nécessaire.
- Avec plusieurs paramètres, **la clé « valeur » est nécessaire**

ROUTING



REQUÊTE HTTP

- **Request Line:**
  - Contient la méthode (GET, POST, PUT…), le chemin vers la ressource, et la version d' HTTP.
  - Exemple: `GET /index.html HTTP/1.1`
- **Request Headers:**
  - Fournit des metadata sur la requête.

```
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

- **Request Body (optional):**
  - Utilisé pour envoyé des données.
  - Exemple d'une requête POST :

```
username=john&password=1234
```

ROUTING: DÉFINITION D'UN CONTRÔLEUR

- Toutes les méthodes à associer à des routes doivent être implémentés dans une **classe annotée par une annotation de contrôleur** :
  - `@RestController`
    Pour les contrôleurs, renvoyant des données (textes, XML, JSON…)
  - `@Controller`
    Pour les controllers renvoyant des pages web (fichiers HTML ou construction de page HTML avec un moteur de rendu).

```
@RestController
public class MyControler {

   @RequestMapping("/")
   public String welcome() {
      return "Hello World";
   }
}
```

ROUTING

Serveur lancé sur http://localhost:8080

- Ensuite les méthodes ou la classe vont être annotée pour spécifier quelle requête elles capturent.
- `@RequestMapping("/")` sur une méthode va lier l'appel de cette méthode sur l'adresse **localhost:8080/**
- `@RequestMapping("/jean")` sur une méthode va lier l'appel de cette méthode sur l'adresse **localhost:8080/jean**
- `@RequestMapping("/jean")` est similaire à `@GetMapping("/jean")`
- D'autres annotations existent `@PostMapping, @PutMapping` …

ROUTING

GET **localhost:8080/**

GET **localhost:8080/conferences/**

GET **localhost:8080/conferences/first/**

```
@Controller
public class ConferenceController {

    @GetMapping("/")
    public String getConferences() {
        return "conferences";
    }

    @GetMapping("/conferences")
    public String addUserToConference() {
        return "conferences";
    }

    @GetMapping("/conferences/first")
    public String getAllConferences1() {
        return "First conferences";
    }
}
```

ROUTING : FACTORISATION

GET **localhost:8080/users/**

GET **localhost:8080/users/first/**

```
@Controller
@RequestMapping("/users")
public class UserController {

    @GetMapping("/")
    public String getUsers() {
        return "users";
    }

    @GetMapping("/first")
    public String getFirstUser() {
        return "first user";
    }
}
```

ROUTING

GET **localhost:8080/users/**

GET **localhost:8080/users/first/**

POST **localhost:8080/users/first/**

```
@RestController
@RequestMapping("/users")
public class UserControleur {

    @GetMapping("/")
    public String getUsers() {
       return "users";
   }

    @GetMapping("/first")
    public String getFirstUser() {
       return "first user";
   }

   @PostMapping("/first")
    public String postFirstUser() {
       return "POST first user";
   }
}
```

PATHVARIABLE

- Pour **relier 1 méthode à plusieurs URL**, on peut utiliser une `PathVariable`:

```
@RequestMapping("/hello/{name}")
public String welcome(@PathVariable String name) {
    return "Hello " + name;
}
```

- cf. PathVariable in https://www.baeldung.com/spring-requestmapping

ROUTING : PATHVARIABLE

GET **localhost:8080/users/**

GET **localhost:8080/users/first/**

GET **localhost:8080/users/number/first**

GET **localhost:8080/users/number/second**

GET **localhost:8080/users/number/third**

```
@Controller
@RequestMapping("/users")
public class UserController {

    @GetMapping("/")
    public String getUsers() {
        return "users";
    }

    @GetMapping("/first")
    public String getFirstUser() {
        return "first user";
    }

    @GetMapping("/number/{nb}")
    public String getNbUser(@PathVariable
String nb) {
        return nb + " user";
    }
}
```

ROUTING : REQUESTPARAM

- Gérer les URLs avec paramètres :
  http://localhost:8080/hello?name=Jean

```
@RequestMapping("/hello")
public String welcomeName(@RequestParam String name) {
    return "Hello " + givenName + " " + name;
}
```

- **Autant de RequestParam que nécessaires peuvent être utilisés.**
  Avec une méthode **POST**, les paramètres sont passés dans le body du protocole HTTP et sont gérés de la même manière.

- Ils peuvent être combinés avec @PathVariable

ROUTING : REQUESTPARAM

- **http://localhost:8080/hello?name=Jean**
- Le nom du paramètre de la méthode welcomeName doit être le même que celui dans l'URL.

```
@RequestMapping("/hello")
public String welcomeName(@RequestParam String name) {
    return "Hello " + name;
}
```

- Variantes : si on souhaite que le nom de la variable Java soit différente.

```
@RequestMapping("/hello")
public String welcomeName(@RequestParam("name") String n) {
    return "Hello " + n;
}
```

- Plus d'options :

```
@RequestParam(value = "name", defaultValue = "test")
```